

← Go Back

Hardware

Nano RP2040 Connect ▾

Tutorials

Nano RP2040 Connect
User Manual

Web Server AP Mode with
Arduino Nano RP2040
Connect

BLE Device to Device with
Nano RP2040 Connect

Nano RP2040 Connect
Chromebook Setup

Nano RP2040 Datalogger
with MicroPython

Using the IMU Machine
Learning Core Features

Accessing IMU Data on
Nano RP2040 Connect

Setting up Nano RP2040
Connect with Arduino
Cloud

Reading Microphone Data
on Nano RP2040 Connect

MicroPython and the IMU
Machine Learning Core
Features

Getting Started With the
Nano RP2040 Connect and
OpenMV

Control Built-in RGB LED
over Wi-Fi with Nano
RP2040 Connect

Voice Commands With The
Arduino Speech
Recognition Engine

Home / Hardware / Nano RP2040
Connect / Nano RP2040 Connect
User Manual

i This tutorial refers to a
product that has reached
its end-of-life status.

Nano RP2040
Connect User
Manual

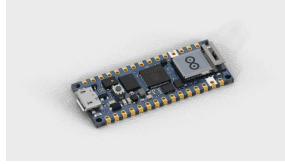
Learn how to set up the Nano RP2040
Connect, get a quick overview of the
components, information regarding
pins and how to use different Serial
(SPI, I2C, UART) and Wireless (Wi-Fi,
Bluetooth®) protocols.

Author · Karl Söderby · Last
revision · 09/30/2025

ON THIS PAGE

Board Package

- Datasheet
- Installation +
- Arduino Cloud
- Upgrading the Firmware
- Bootloader +
- Pins +
- IMU +
- Microphone +
- RGB
- Communication +
- Connectivity
- Wi-Fi +
- Bluetooth® +
- USB Keyboard



The Arduino Nano
RP2040 Connect

The Arduino® Nano RP2040 Connect is a development board in Nano format, based on the [RP2040](#) microcontroller. It features a Wi-Fi / Bluetooth® module, a 6-axis IMU (Inertial Measurement Unit) with machine learning capabilities, a microphone and a built-in RGB.

This article is a collection of guides, API calls, libraries and tutorials that can help you get started with the Nano RP2040 Connect board.

You can also visit the [documentation platform for the Nano RP2040 Connect](#).

Board Package

The Nano RP2040 Connect uses the [Arduino Mbed OS Nano Board Package](#).

Datasheet

The full datasheet is available as a downloadable PDF from the link below:

[Download the Arduino Nano RP2040 Connect datasheet](#)

Installation

Arduino IDE 1.8.X

The Nano RP2040 Connect can be programmed through the **Classic Arduino IDE 1.8.X**. To install your board, you can check out the guide below:

[Installing the Arduino Mbed OS Nano Board Package](#)

Arduino IDE 2

The Nano RP2040 Connect can be programmed through the **Arduino IDE 2**. To install your board, you can check out the guide below:

[How to use the board manager with the Arduino IDE 2](#)

Cloud Editor

The Nano RP2040 Connect can be programmed through the **Cloud Editor**. To get started with your board, you will only need to install a plugin, which is explained in the guide below:

[Getting started with the Cloud Editor](#)

Arduino Cloud

The Nano RP2040 Connect is compatible with the [Arduino Cloud](#).

you to create IoT applications in just minutes.

If you need help to get started, you can go through the [Nano RP2040 Connect with Arduino Cloud tutorial](#).

Upgrading the Firmware

If you need to upgrade the firmware on your Nano RP2040 Connect, you can do so by using the Arduino IDE 2. See the [link](#) below for a more detailed guide.

[Upgrading Nano RP2040 Connect NINA firmware](#)

Bootloader

Since our upload procedure relies on the Raspberry's bootloader using a mass storage device, if your computer is fast enough during an upload, it can notify you about an USB removable being plugged.

When a sketch is uploaded successfully, the mass storage of the Nano RP2040 Connect may be visible in the operating system. The mass storage should only appear for a few seconds, then it will automatically close. When this occurs, we can force the ROM bootloader mode, which will enable mass storage, allowing us to upload UF2 images like CircuitPython / MicroPython or a

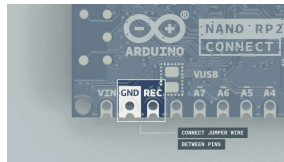
Forcing Bootloader

There is a risk that the uploading process gets stuck during an upload. If this happens, we can double-tap the reset button, to forcefully trigger the bootloader.

Board Not Detected

Sometimes the board is not detected even when the board is connected to your computer. This can be solved through the following steps:

1. Connect the board to your computer via USB.
2. Place a jumper wire between the REC and GND pins on the board, then press the reset button.

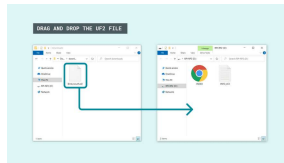


Place a jumper wire between REC and GND pins.

3. This will open the mass storage device. You can now remove the jumper wire.
4. Upload a basic sketch, such as the `blink` example to the board (even though it is not visible in the port selection).
5. When it has finished uploading, your board should be visible in the board/port selection, and your board's built-in LED should be

Alternatively, you can choose to factory-reset the board by dragging the `blink.ino.elf.uf2` file into the mass storage. You can download the file from the link below:

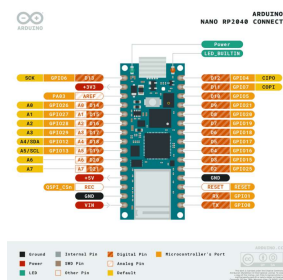
[Download the blink.ino.elf.uf2 file](#)



Drag and drop the `blink.ino.elf.uf2` file into RP2040's mass storage.

After dragging the `U2F` file, the board will be flashed with a program that blinks the built-in LED, and shifts between the `red`, `green` and `blue` pixels.

Pins



The pinout for Nano RP2040 Connect.

Analog Pins

The Nano RP2040 Connect has 8 analog pins, that can be used through the `analogRead()`

```
1 value = analogRead(pin,
```

Please note: pin `A4` and `A5` should be used for I2C only.

Please note: pin `A6` and `A7` does not support PWM.

PWM Pins

Most of the digital & analog pins can be used as PWM (Pulse Width Modulation) pins, the exception being the following pins:

A4
A5
A6
A7

```
1 analogWrite(pin, value)
```

Digital Pins

There are a total of 14 digital pins, whereas the 8 analog pins can also be used as digital pins.

Please note: A4 and A5 are I2C only, while A6 and A7 can only be used as inputs.

To use them, we first need to define them inside the `void setup()` function of our

Note: digital pin 3 cannot be configured as

`INPUT_PULLUP`.

```
1 pinMode(pin, INPUT); //
2 pinMode(pin, OUTPUT); /
3 pinMode(pin, INPUT_PULL
```

To read the state of a digital pin:

```
1 state = digitalRead(pin
```

To write a state to a digital pin:

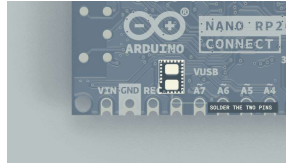
```
1 digitalWrite(pin, HIGH)
```

5V Pin

The Arduino RP2040 Connect operates at 3.3 V, and has the 5V pin (VUSB) disabled by default. This is a **safety precaution**, as connecting higher voltage signals to the board can damage the hardware.

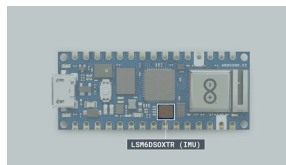
The 5V pin will be **enabled** if the pads marked VUSB are shorted, by soldering them.

The VUSB pin is located on the bottom of the board. The pads on the Arduino RP2040 Connect are highlighted below.



Soldering the VUSB pins.

IMU



The LSM6DSOXTR sensor

LSM6DSOXTR

The LSM6DSOXTR from STM is an IMU (Inertial Measurement Unit) that features a 3D digital **accelerometer** and a 3D digital **gyroscope**. It features among many other things, a **machine learning core**, which is useful for any motion detection projects, such as free fall, step detector, step counter, pedometer.

This module also features an embedded **temperature sensor**.

LSM6DSOX Library

To access the data from the LSM6DSOX module, we need to install the [LSM6DSOX](#) library, which comes with examples that can be used directly with the Nano RP2040 Connect.

It can be installed directly from the library manager through the IDE of your choice. To use it, we need to include at the top of the sketch:

```
1 #include <Arduino_LSM6D
```

And to initialize the library, we can use the following command inside

```
void setup() {
```

```
1 if (!IMU.begin()) {  
2     Serial.println("Fail  
3     while (1);  
4 }
```

Accelerometer

The accelerometer data can be accessed through the following commands:

```
1 float x, y, z;  
2  
3 if (IMU.accelerationAvailable()  
4     IMU.readAcceleration(x, y, z);  
5 }
```

Gyroscope

The gyroscope data can be accessed through the following commands:

```
1 float x, y, z;  
2  
3 if (IMU.gyroscopeAvai  
4     IMU.readGyroscope(x  
5 }
```

Temperature

The temperature data can be accessed through the following code:

```
1 if (IMU.temperatureAvai  
2 {  
3     int temperature_deg  
4     IMU.readTemperature  
5  
6     Serial.print("LSM6D  
7     Serial.print(temper  
8     Serial.println(" °C  
9 }
```

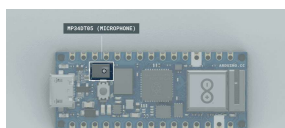
Tutorials

If you want to learn more on how to use the IMU, please check out the tutorial below:

[Accessing IMU data with Nano RP2040 Connect](#)

[Using the IMU Machine Learning Core Features](#)

Microphone



The MP34DT06JTR
microphone sensor.

MP34DT06JTR

The **MP34DT06JTR** is a compact, low-power omnidirectional digital MEMS microphone with an IC interface. It has a 64 dB signal-to-noise ratio, is capable of sensing acoustic waves and can operate in temperatures of -40 °C to +85 °C.

PDM Library

To access the data from the MP34DT06JTR, we need to use the [PDM](#) library that is included in the **Arduino Mbed OS Nano Board Package**. If the Board Package is installed, you will find an example that works by browsing **File > Examples > PDM > PDMSerialPlotter**.

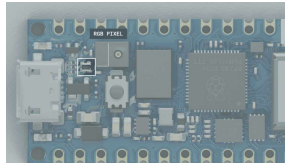
Please note: The sampling frequency in the PDMSerialPlotter example is set to 16000 Hz. If the microphone appears to not be working (monitor is printing a value of -128), try to change this rate to 20000 Hz. You can change this at the top of the PDMSerialPlotter example sketch, as shown in the example below:

```
1 static const int freque
```

If you want to learn more on how to use the Microphone, please check out the tutorial below:

[Reading microphone data with Nano RP2040 Connect](#)

RGB



The RGB pixel.

Please note: While using the Bluetooth® Low Energy mode on the NINA module, the RGB cannot be used by default. While the module is in Bluetooth® Low Energy mode, SPI is deactivated, which is used to control the RGBs.

The Nano RP2040 Connect features a built-in RGB that can be utilized as a feedback component for applications. The RGB is connected through the W-102 module, so the `WiFiNINA` library needs to be installed and included at the top of your sketch to work.

The `Wi-FiNINA` library is required to use the RGB.

```
1 #include <Wi-FiNINA.h>
```

The pins needs to be defined

```
1 pinMode(LED_R, OUTPUT);  
2 pinMode(LED_G, OUTPUT);  
3 pinMode(LED_B, OUTPUT);
```

To turn ON the pixels, write a

`HIGH` state to the LED:

```
1 digitalWrite(LED_R, HIGH)  
2 digitalWrite(LED_G, HIGH)  
3 digitalWrite(LED_B, HIGH)
```

To turn OFF the pixels, write a

`LOW` state to the LED:

```
1 digitalWrite(LED_R, LOW)  
2 digitalWrite(LED_G, LOW)  
3 digitalWrite(LED_B, LOW)
```

We can also choose a value
between 255 - 0 to write to the
LED:

```
1 analogWrite(LED_R, 72);  
2 analogWrite(LED_G, 122);  
3 analogWrite(LED_B, 234);
```

Communication

Like other Arduino® products, the Nano RP2040 Connect features dedicated pins for different protocols.

The pins used for SPI (Serial Peripheral Interface) on the Nano RP2040 Connect are the following:

(CIPO) - D12

(COPI) - D11

(SCK) - D13

(CS) - Any GPIO (except for A6/A7)



The signal names MOSI, MISO and SS has been replaced by COPI (Controller Out, Peripheral In), CIPO (Controller In, Peripheral Out) and CS (Chip Select).

To use SPI, we first need to include the **SPI** library.

```
1 #include <SPI.h>
```

Inside `void setup()` we need to initialize the library.

```
1 SPI.begin();
```

And to write to the device:

```
1 digitalWrite(chipSelect
2
3 SPI.transfer(address)
4 SPI.transfer(value);
5
6 digitalWrite(chipSele
```

The pins used for I2C (Inter-Integrated Circuit) on the Nano RP2040 Connect are the following:

(SDA) - A4

(SCL) - A5

To use I2C, we can use the [Wire](#) library, which we need to include at the top of our sketch.

```
1 #include <Wire.h>
```

Inside `void setup()` we need to initialize the library.

```
1 Wire.begin();
```

And to write something to a device connected via I2C, we can use the following commands:

```
1 Wire.beginTransmission(  
2   Wire.write(byte(0x00))  
3   Wire.write(val); //se  
4   Wire.endTransmission(  
◀────────────────────────▶
```

UART

The pins used for UART (Universal asynchronous receiver-transmitter) are the following:

(Rx) - D0

(Tx) - D1

To send and receive data through


```
1 Serial1.begin(9600);
```

To read incoming data, we can use a `while` loop() to read each individual character and add it to a string.

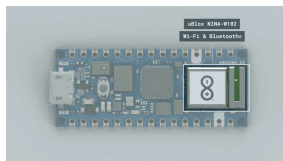
```
1 while(Serial1.available
2     delay(2);
3     char c = Serial1.re
4     incoming += c;
5 }
```

And to write something, we can use the following command:

```
1 Serial1.write("Hello wo
```

Connectivity

The Nano RP2040 Connect supports both Wi-Fi and Bluetooth® through the [uBlox W-102](#) module. To use this module, we can use the [WiFinINA](#) library or the [ArduinoBLE](#) library.



Wi-Fi & Bluetooth®
module.

Wi-Fi

To use the Wi-Fi module on the Nano RP2040 Connect, we will need to install the [WiFiNINA](#) library, and include it at the top of our sketch:

```
1 #include <WiFiNINA.h>
```

To connect to a Wi-Fi network, we can use the following command:

```
1 WiFi.begin(ssid, pass);
```

GET / POST Requests

The **WiFiNINA** library can be used to make GET & POST requests, while connected to the server. The command below is used to connect to `www.google.com` and return the results of searching for the keyword `arduino`.

```
1 if (client.connect(serv
2     client.println("GET
3     client.println("Hos
4     client.println("Con
5     client.println();
6 }
```

Tutorials

Connect

RGB Control over Wi-Fi with Arduino Nano RP2040 Connect

Bluetooth®

Please note: While using the Bluetooth® Low Energy mode on the NINA module, the RGB cannot be used by default. While the module is in Bluetooth® Low Energy mode, SPI is deactivated, which is used to control the RGBs.

To enable Bluetooth® on the Nano RP2040 Connect, we can use the [ArduinoBLE](#) library, and include it at the top of our sketch:

```
1 #include <ArduinoBLE.h>
```

Set the service and characteristic:

```
1 BLEService ledService("
2 BLEByteCharacteristic s
```

Set advertised name and service:

```
1 BLE.setLocalName("Nano
2 BLE.setAdvertisedServ
```

```
1 BLE.advertise();
```

Listen for BLE peripherals to connect:

```
1 BLEDevice central = BLE
```

Tutorials

[Nano RP2040 device to device via Bluetooth®](#)

USB Keyboard

To use the board as a keyboard, you can refer to the [USBHID](#) library that can be found inside the Board Package.

You first need to include the libraries and create an object:

```
1 #include "PluggableUSBH
2 #include "USBKeyboard.h
3
4 USBKeyboard Keyboard;
```

Then use the following command to write something:

```
1 Keyboard.printf("This i
```

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

